

Comparing Threads in C and Rust

Josiah Lawrence

CSU Chico

CSCI 440: Operating Systems

Dr. Bryan Dixon

May 13, 2026

Comparing Threads in C and Rust

This project investigates the difference in time performance of threaded programs in C and Rust. If it could be shown that near-identical threaded programs run significantly faster in either language, this information would be useful for developers of new projects in deciding which language is best for their use case. To answer this question, I tested six programs – three test cases, each written in C and Rust – in a specially designed environment. The test cases include matrix multiplication, Monte Carlo estimation of pi, and DNS lookup. These were chosen because they are mostly CPU-dependent, making them ideal for testing thread performance.

The experimental setup is intended to reduce confounding factors and maintain a consistent environment for each program. All data was collected on a laptop with 16 CPU cores and 32 GB of memory. I anticipated two main confounding variables: scheduling inconsistency due to background processes, and unpredictable network conditions affecting the DNS lookup test case. To limit background processes, I used a minimal bare-metal NixOS installation with a headless configuration. This eliminated most background noise in the OS. Creating a consistent environment for the DNS case was more difficult. To achieve maximally consistent network throughput, I used a direct Ethernet connection to a dedicated router/modem with no other devices on the network. In addition, I set up a local DNS resolver on the test machine to get more fine-grained control over the cache. Since the goal of this experiment is to measure thread performance, the local resolver is configured to minimize cache misses. Notably, it is set to retain cache records for at least 24 hours. This potentially sacrifices record accuracy but ensures that all records are retained by the cache after the warmup runs.

The testing procedure is executed by several Python scripts. The first, a master script, runs an array of six experimental scripts in turn, one for each program. The experimental scripts are almost identical, with some differences for the DNS lookup case. First, a list of workload sizes is defined. After compiling the program, the script executes three warmup runs, which help to keep the data consistent. In the case of the DNS lookup program, the warmup consists of running the program on all of the hostnames to be used in the experiment, ensuring all records are cached and preventing the program from spending the majority of its runtime blocking for DNS queries. The program is then executed a number of times n , such that the margin of error (ME) is lower than 5% of the mean runtime, with a confidence level of 95% ($Z = 1.96$). For the DNS lookup case, the error threshold was 10% of the mean because this program tended to have extremely variable runtimes. For the same reason, the DNS lookup program runs for a minimum n

of 100 and a maximum n of 1000, which prevents outliers from causing the experiment to run for misleadingly low or explosively high values of n . The runtimes of the threaded portion of the program are measured within the program itself and recorded in a CSV file. Then, the master script runs the next experimental script. After all programs are tested, charts are generated using Matplotlib.

The test results included in the Tables and Figures section show conflicting differences between the two languages. In the matrix multiplication case, the C implementation runs slightly faster than the Rust version for all workload sizes. But in the Monte Carlo test case, the Rust implementation excels by an order of magnitude. This may suggest that threaded programs are more efficient in Rust. Both the matrix multiplication and Monte Carlo have a margin of error equal to 5% of the mean runtime, and a confidence value of 95% ($Z = 1.96$). Unfortunately, the data for the DNS lookup case is skewed by numerous large outliers. Due to technical difficulties and time constraints, I have been unable to eliminate the noise from these data.

Based on these test results, there is not enough conclusive evidence to show a significant difference in the time performance of threaded programs in C and Rust. Due to the discrepancy between the matrix multiplication and Monte Carlo cases, as well as the noisy data from the DNS case, more research is needed to make a definitive conclusion.

Tables and Figures

Table 1: Matrix multiplication in C. Mean runtime and standard deviation.

Size	Mean Runtime (μ s)	Std. Deviation
100	415.6667	7.371115
250	2265.676	336.993905
500	16878.57	1047.319731
750	52409.75	2376.574618
1000	120888.7	4381.858206
1500	465676.3	5878.873815
2000	1695078	20102.902038

Figure 1: Matrix Multiplication in C. Box and whisker plot representing execution time for various workload sizes.

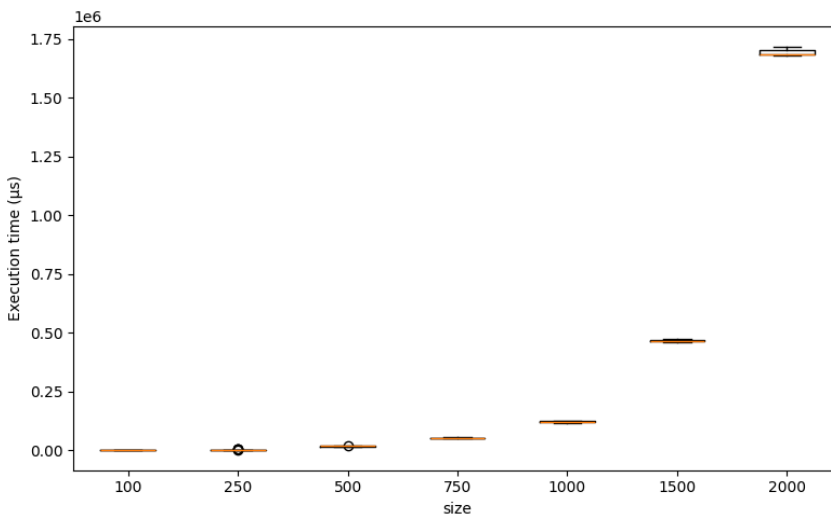


Table 2: Matrix multiplication in Rust. Mean runtime and standard deviation.

Size	Mean Runtime (μ s)	Std. Deviation
100	460.3617	79.101593
250	3258.000	264.058706
500	21349.46	1962.210090
750	61863.40	3234.793780
1000	141334.2	7199.234745
1500	666778.3	4314.472428
2000	2018488	18358.290234

Figure 2: Matrix Multiplication in Rust. Box and whisker plot representing execution time for various workload sizes.

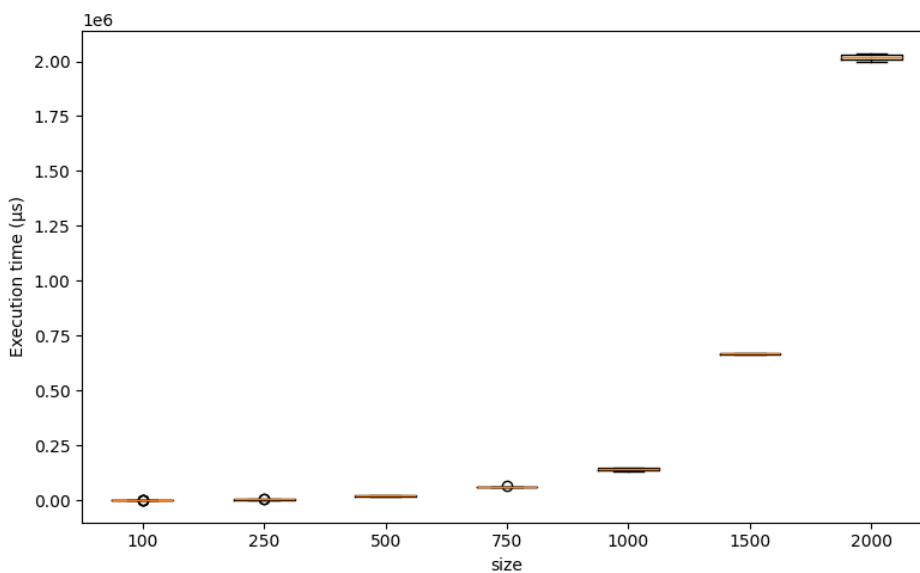


Table 3: Monte Carlo in C. Mean runtime and standard deviation.

Size	Mean Runtime (μ s)	Std. Deviation
10000000	65420.71	3895.386062
50000000	306760.6	21462.872875
100000000	617996.0	49584.528373
500000000	3217947	61851.936124

Figure 3: Monte Carlo in C. Box and whisker plot representing execution time for various workload sizes.

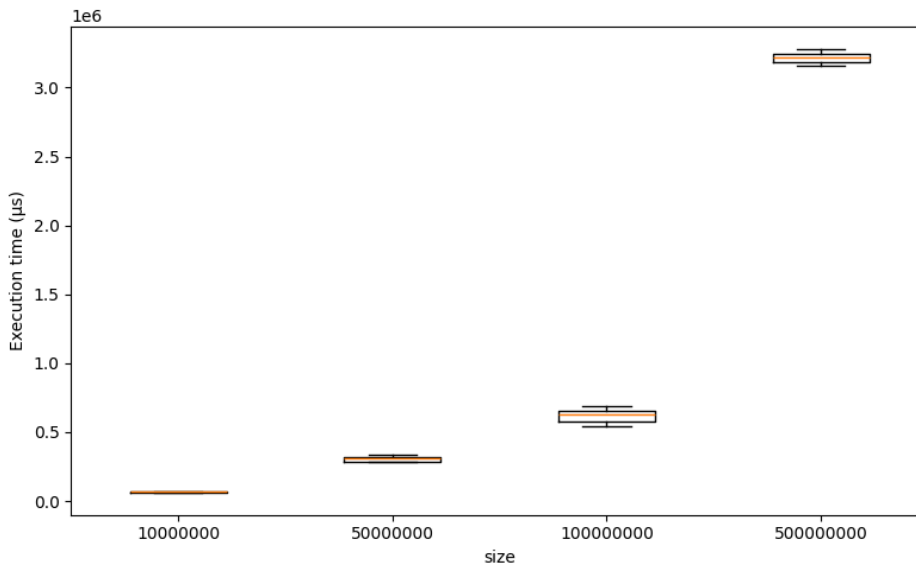


Table 4: Monte Carlo in Rust. Mean runtime and standard deviation.

Size	Mean Runtime (μ s)	Std. Deviation
10000000	4613	91.994565
50000000	17537	1577.423248
100000000	34113	1828.320404
500000000	145272	1984.252588

Figure 4: Monte Carlo in Rust. Box and whisker plot representing execution time for various workload sizes.

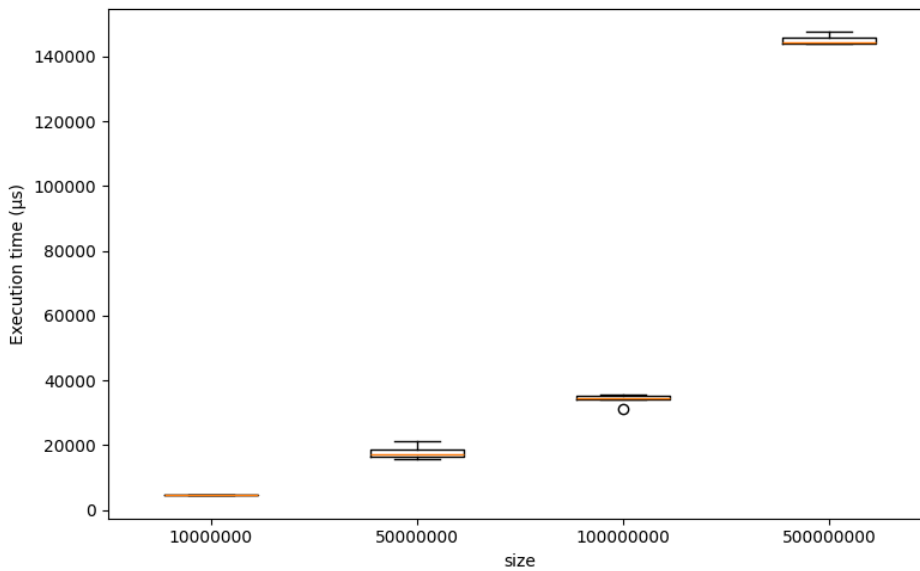


Table 5: DNS lookup in C. Mean runtime and standard deviation.

Size	Mean Runtime (μ s)	Std. Deviation
100	383616.573	3.057058e+06
250	333716.630	2.871206e+06
500	1538570.930	5.991785e+06

Figure 5: DNS Lookup in C. Box and whisker plot representing execution time for various workload sizes.

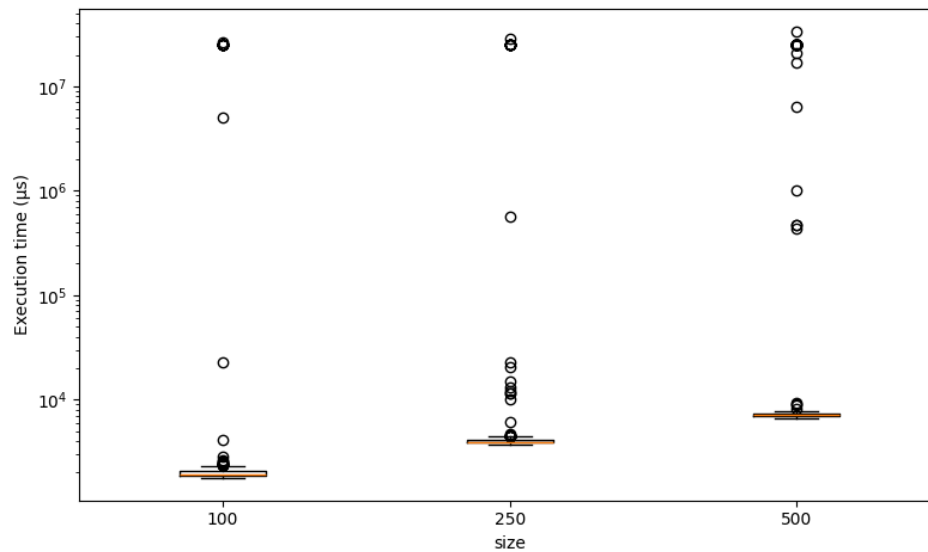


Table 6: DNS lookup in Rust. Mean runtime and standard deviation.

Size	Mean Runtime (μ s)	Std. Deviation
100	123632.9	1708948
250	247896.5	2420824
500	246961.7	2618175

Figure 6: DNS Lookup in Rust. Box and whisker plot representing execution time for various workload sizes.

